

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

FSI308-R

Landing zone patterns and anti-patterns: Balance agility and safety

Kenneth Jackson

Senior Manager Solution Architecture
AWS Global Financial Services

Zahi Ben Shabat

Principal Prototype Solution Architect
AWS Global Financial Services



Agenda

1. What is a landing zone?
2. Mental model for landing zones
3. Patterns and anti-patterns
4. Developer experience at scale
5. Q&A

What is a landing zone?

“A **landing zone** is a well-architected, **multi-account** AWS environment that is **scalable** and **secure**. This is a **starting point** from which your organization can **quickly deploy** applications with confidence in your security and infrastructure environment.”

- AWS Prescriptive Guidance

This is your session: What's on your mind?



Mental model for landing zones

Why are you moving to AWS?



Return on investment (ROI)

241% ROI
over 3 years*



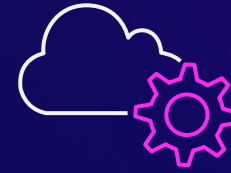
Staff productivity

62% more
efficient IT
infrastructure
staff



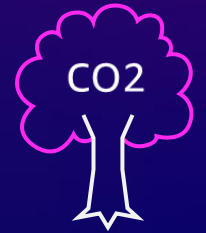
Operational resilience

57% decrease
in downtime



Business agility

37% decrease
in time
to market



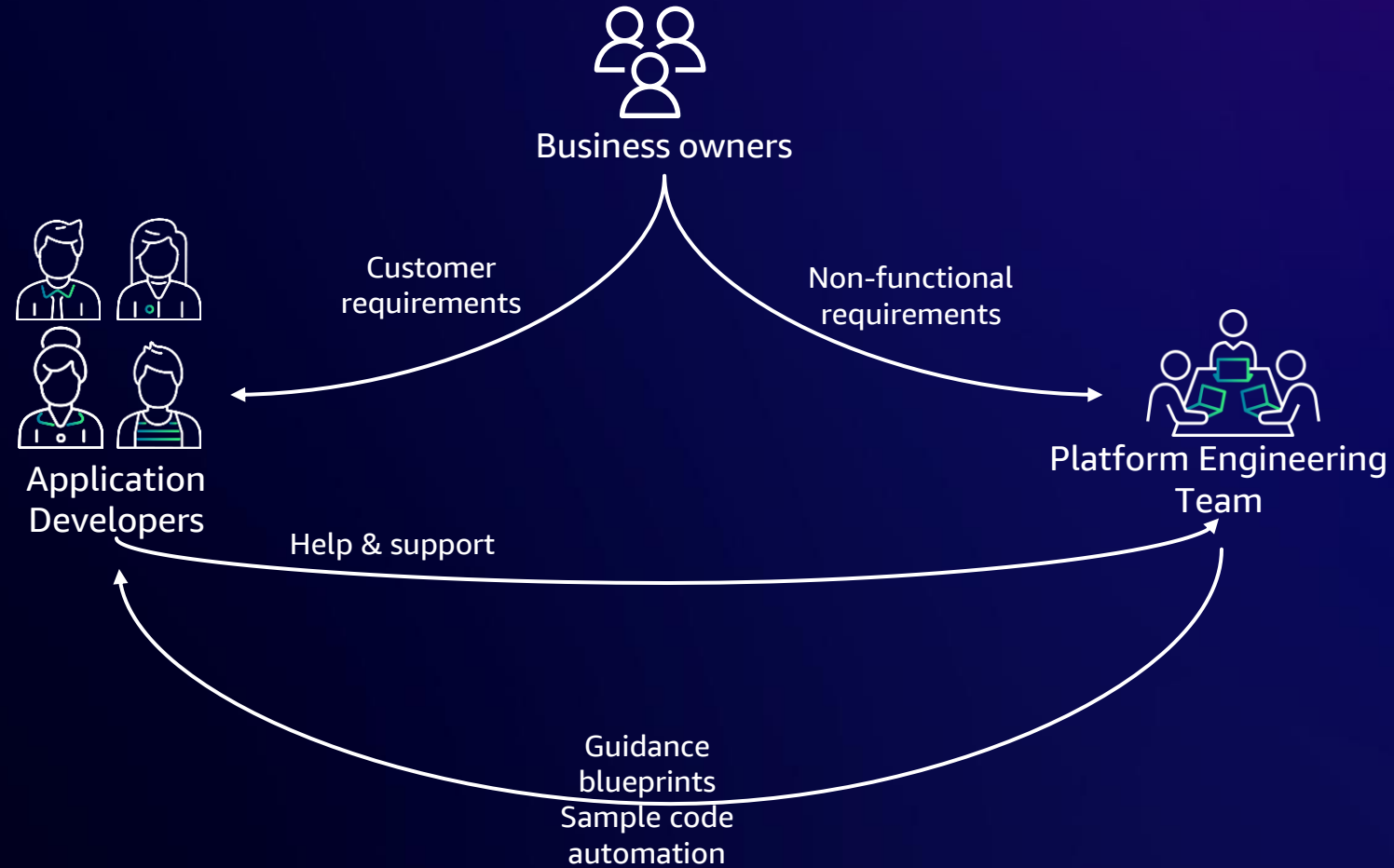
Carbon savings

88% reduction
in carbon
footprint of
IT operations

Based on independent research of 1,500 AWS enterprise customers

*Source: "The Total Economic Impact of AWS Cloud Operations: Cost Savings And Business Benefits Enabled By Operation on AWS. May 2022. A commissioned study conducted by Forrester Consulting on behalf of Amazon Web Services."

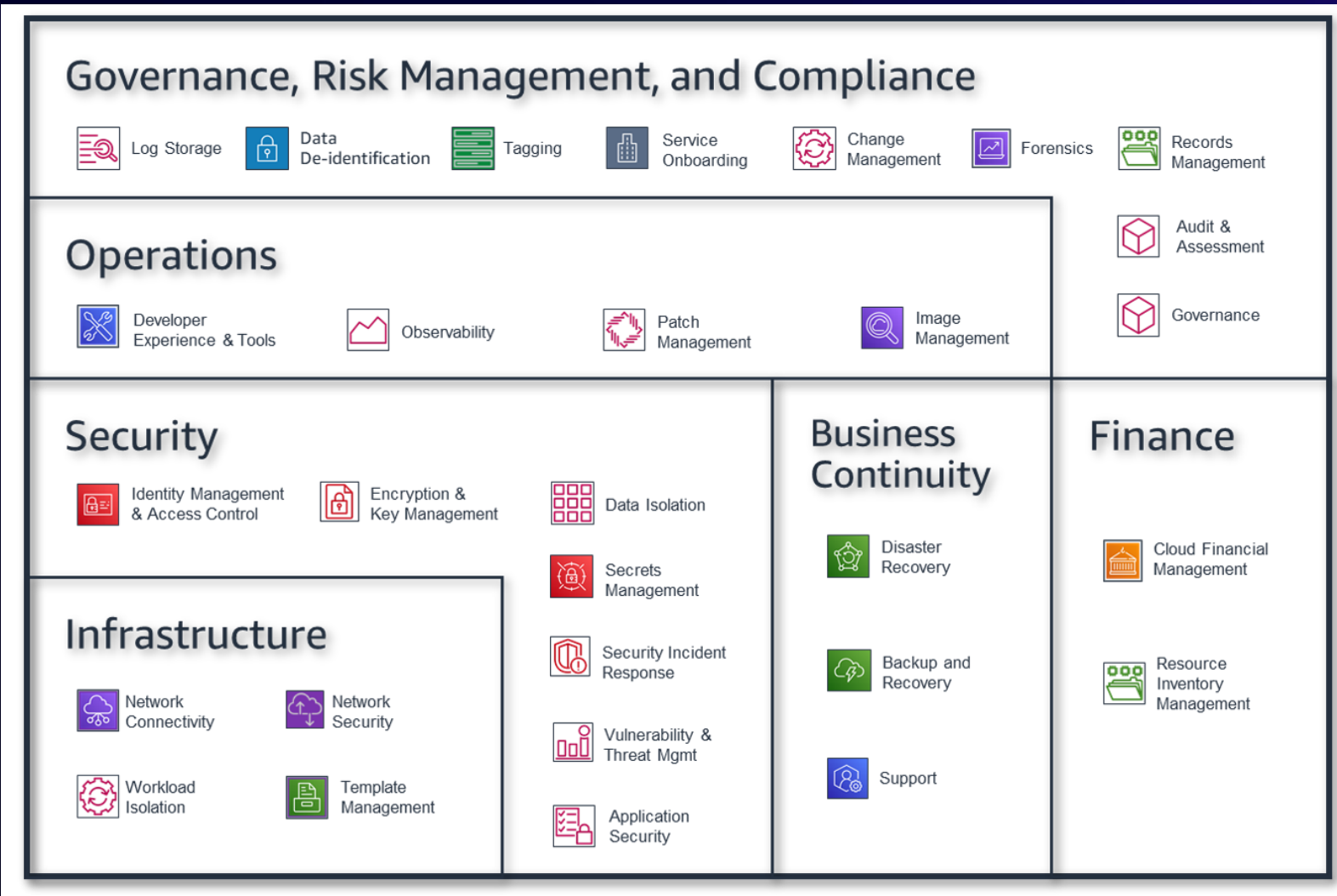
Enterprise personas



Challenge 1: Platform engineers are scarce



Challenge 2: Many capabilities to engineer



Questions:

Where do I start?

How comprehensive do these capabilities need to be?

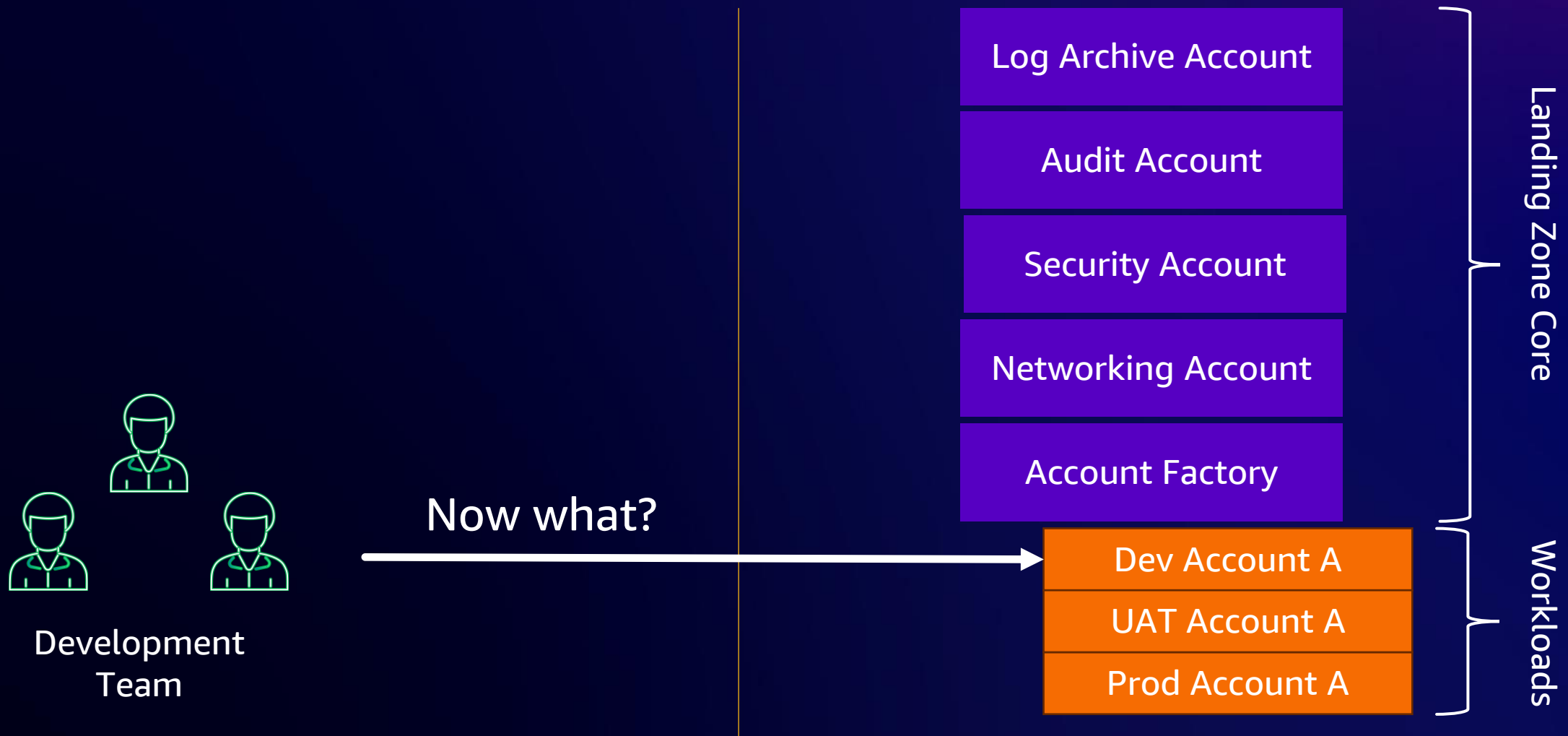
Centralized or line-of-business specific?

<https://aws.amazon.com/architecture/cloud-foundations/>

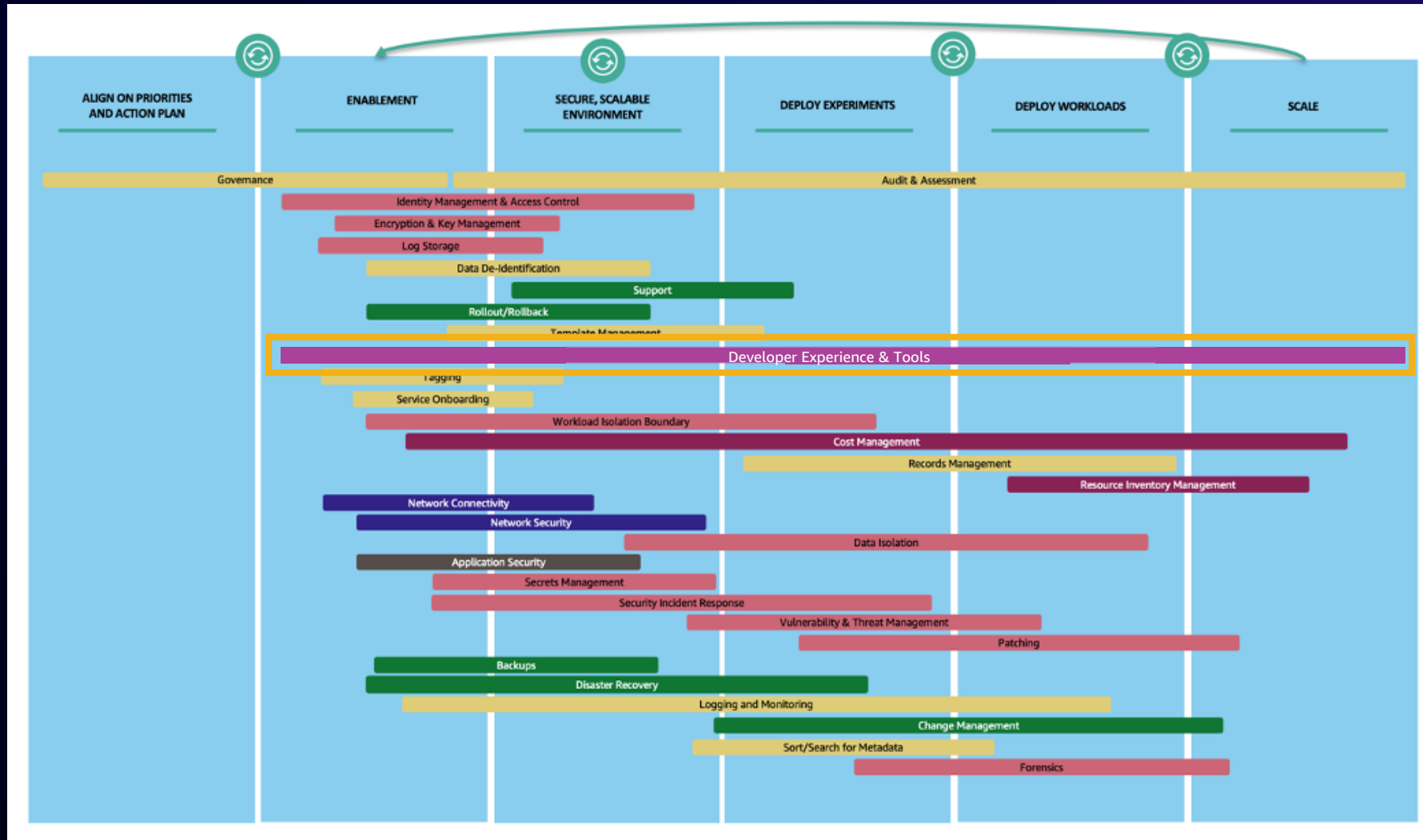


© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Challenge 3: Developers have varied cloud skill



Mental model: Start with developer experience



Work backwards from your developer skillset and application patterns

Engineer the minimum viable foundational capabilities

Continually evolve to support new patterns and increasing cloud expertise

<https://aws.amazon.com/architecture/cloud-foundations/>



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Landing zone patterns and anti-patterns

Patterns and anti-patterns 1

Build it all first

- ✗ Engineer a **custom** solution
- ✗ Build **every capability** before first workload
- ✗ Try to design and accommodate all teams and **all application patterns**
- ✗ Require developers to learn **many new tools** and techniques to deploy
- ✗ Build under **assumptions and hypotheses** of what teams actually need

Make one path work, repeat

- ✓ Use AWS Control Tower as foundation
 - ✓ **Account Factory** – templated account vending by environment
 - ✓ **Controls** – preventative (SCPs), detective (AWS Config), and proactive (Cfn hooks)
 - ✓ **Dashboard** – OUs, Accounts, control status, noncompliant resources
- ✓ Work backwards from a **real application pattern**
 - ✓ - Clear success criteria: app team X can self-serve deploy to dev, uat, prod

Patterns and anti-patterns 2

Cloud foundation is “done”

- ✗ Build the foundation **once** and then ignore it once it is done
- ✗ One-time **cloud foundation project**
- ✗ Assemble a team to build the foundations and **disassemble** them when the foundation completed

Continuous upgrades

- ✓ Cloud operating model is **likely to change** over time
 - ✓ Centralized vs. line-of-business
 - ✓ Critical applications with dedicated platform engineering
- ✓ **Multiple versions** of guardrails, templates will be in production
 - ✓ Define processes for controlled deprecation of n-2 versions
 - ✓ Define processes for recording and checking exceptions via API
 - ✓ Define when to fail vs. warn on variance

Patterns and anti-patterns 3

Policy without code

- ✗ Word documents in a central repository
- ✗ Manual and subjective security reviews
- ✗ Difficult to reason over correctness at scale



re:Invent 2022
[Goldman Sachs \(COP313\)](#)

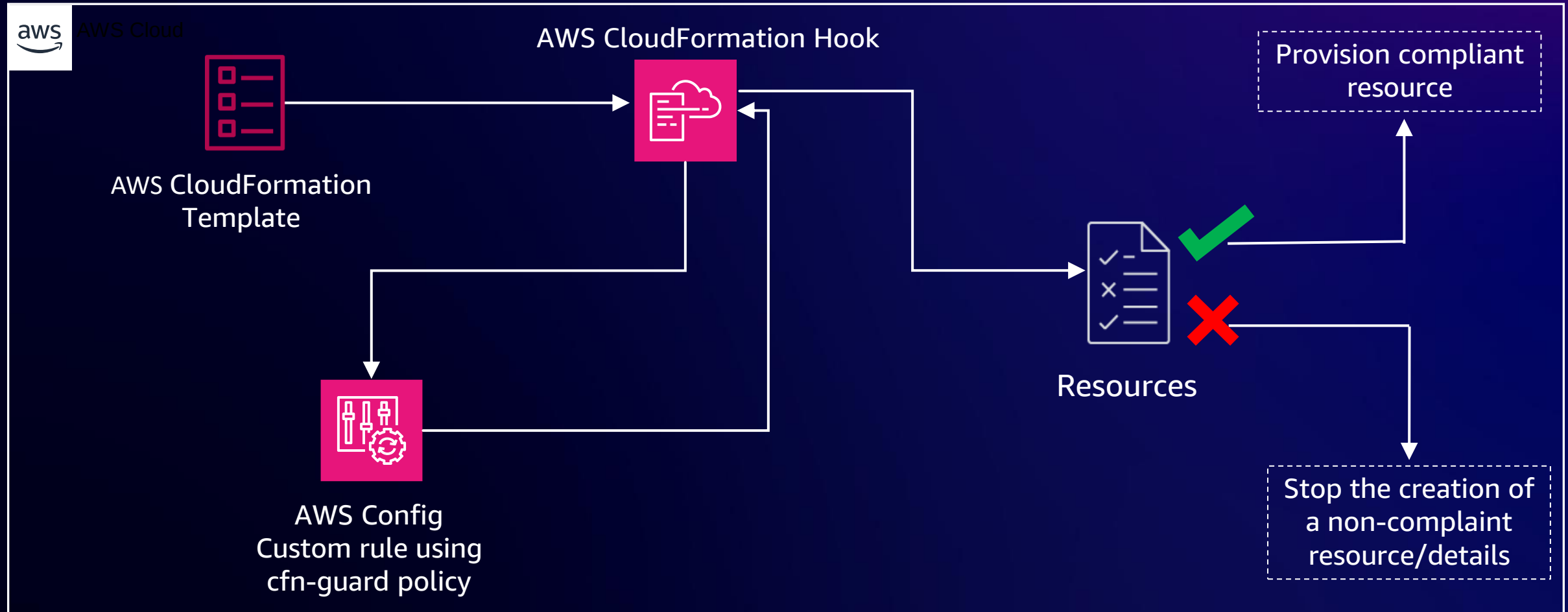


re:Invent 2021
[Policy-as-code workshop](#)

Policy-as-code

- ✓ Human-readable policies in Rego, cfn-guard, Sentinel, checkov, or other languages
- ✓ Continually applied through deployment pipeline or AWS CloudFormation Hooks
- ✓ **Best practices:**
 - ✓ Threat modeling → mitigation objectives → controls-as-code → evidence
 - ✓ Use all control models: preventative, detective, and proactive (shift-left)
 - ✓ Leverage account boundaries as policy simplifiers

Example: Policy-as-code with CloudFormation Hook



Patterns and anti-patterns 4

Neglecting developer experience

- ✗ **Wiki** with instructions and examples
- ✗ Expecting developers to **know AWS** on day 1
- ✗ Ignoring **obstacles** and **lack of standardization**
- ✗ **High cognitive load** on development teams
- ✗ **Ticket backlogs** for platform teams for features



Work backwards from developers

- ✓ Enable "sandbox" accounts for experimentation
- ✓ Support cloud novice and experienced teams
- ✓ Key questions:
 - ✓ Does the DevEx support your goals for moving to cloud?
 - ✓ Does the DevEx match the skillset of the majority of your developers?
 - ✓ Which "paved path" will get the most traffic?
- ✓ Consider "[cloud parties](#)" or other concentrated **learning-by-doing**
- ✓ Forcing function to find and remove friction
- ✓ "Validate and harden blueprints" (JPMC re:Invent 2019 – [FSI204](#))

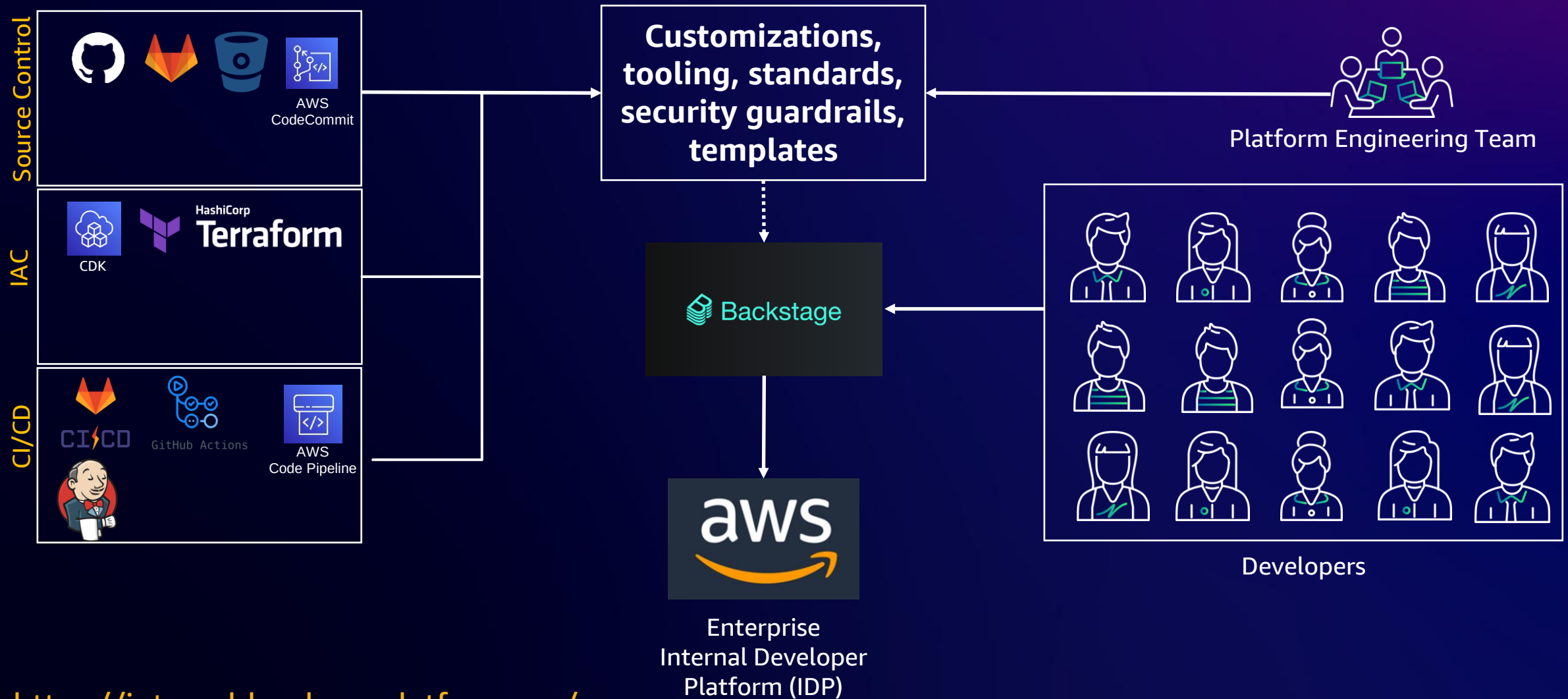
Enabling experimentation & cloud innovation

Environment	Purposes	Networking	Console	Pipeline	Guardrails
Sandbox	Investigating service functionality	Internet egress Corp ingress via proxy Can create VPCs	Read / write access	None	Cost controls Limited restrictions on network & IAM
Development	Ensuring services suitability Ensuring compliance with guardrails Testing new functionality	Internet egress Corporate ingress & egress Pre-configured VPCs Optional VPCE	Read / write access for allowed services	Optional Checks in warn mode	Pre-configured or self-service IAM Detective controls Limited SCPs
UAT	Ensure application will function at scale in prod	Internet egress VPCE mandatory Pre-configured VPCs	Read access Optional breakglass	Mandatory Checks mirror prod	Mirror production
Production	Run the business	Identical to UAT	No standing access	Mandatory	Fail closed

High-quality developer experience at scale

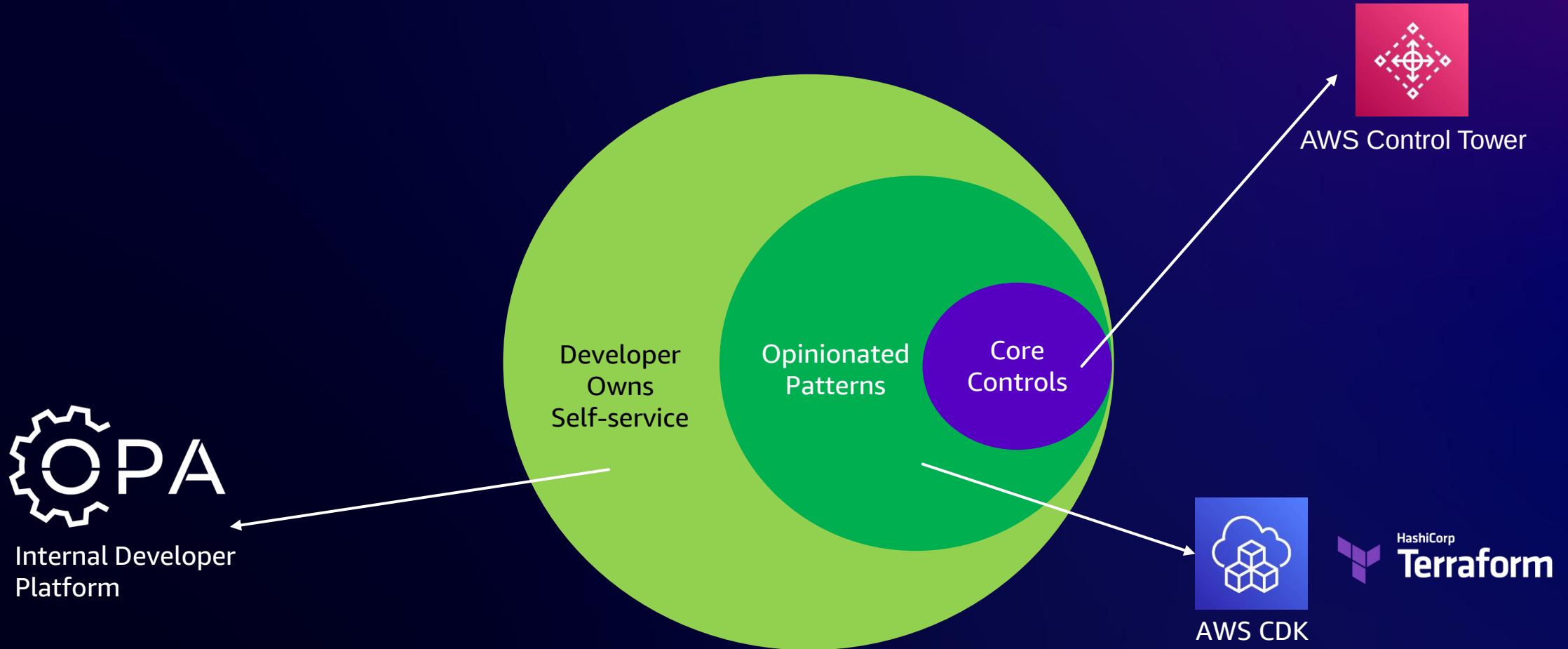


Internal Developer Platforms (IDPs)



<https://internaldeveloperplatform.org/>

Where does it all fit?



Orchestrate Platform and Applications (OPA!)

REFERENCE IMPLEMENTATION OF AN INTERNAL DEVELOPER PLATFORM FOR AWS



Source code



Designed for enterprise



Templates



Example apps



Backstage plugins



Security best practices



<https://OPAonAWS.io/>

More resources

USE QR CODES TO MAKE RESOURCES MORE ACCESSIBLE DURING YOUR SESSION



OPA on AWS



ENT323 - Build
for cloud with
AWS Migration
Hub Refactor
Spaces and
Backstage



Solutions for cloud
foundations



CNCF
platform whitepaper

Thank you!

Kenneth Jackson

jaknn@amazon.com

Zahi Ben Shabat

zshabat@amazon.com



Please complete the session
survey in the mobile app

Reference architecture

